

PRODUCT FEATURE & SWAGGER API GUIDE

Library Management Platform

A complete walkthrough of catalog management, copy inventory, member operations, self-service borrowing, returns, overdue fines, availability, security, and API integration.

React + TypeScript

FastAPI

PostgreSQL

JWT + RBAC

gRPC

Behave BDD

LIBRARY

One platform for the complete circulation lifecycle

Catalog

Authors, books, ISBN metadata, title search, and author-filtered views.

Inventory

Barcode-level physical copies, statuses, total counts, and live availability.

Members

Role-controlled registration, searchable member directory, and account linking.

Loans

Availability-aware borrowing, due dates, personal history, and one-minute refresh.

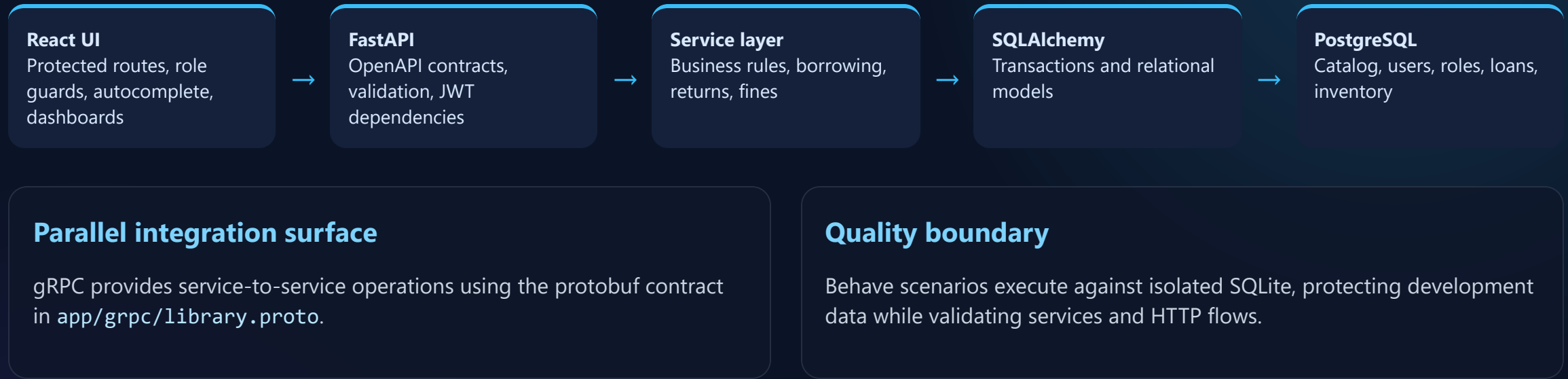
Returns

Self-service returns that finalize fines and restore copy availability atomically.

Security

JWT bearer tokens, protected endpoints, record ownership, and role authorization.

Clear separation from user experience to persistence



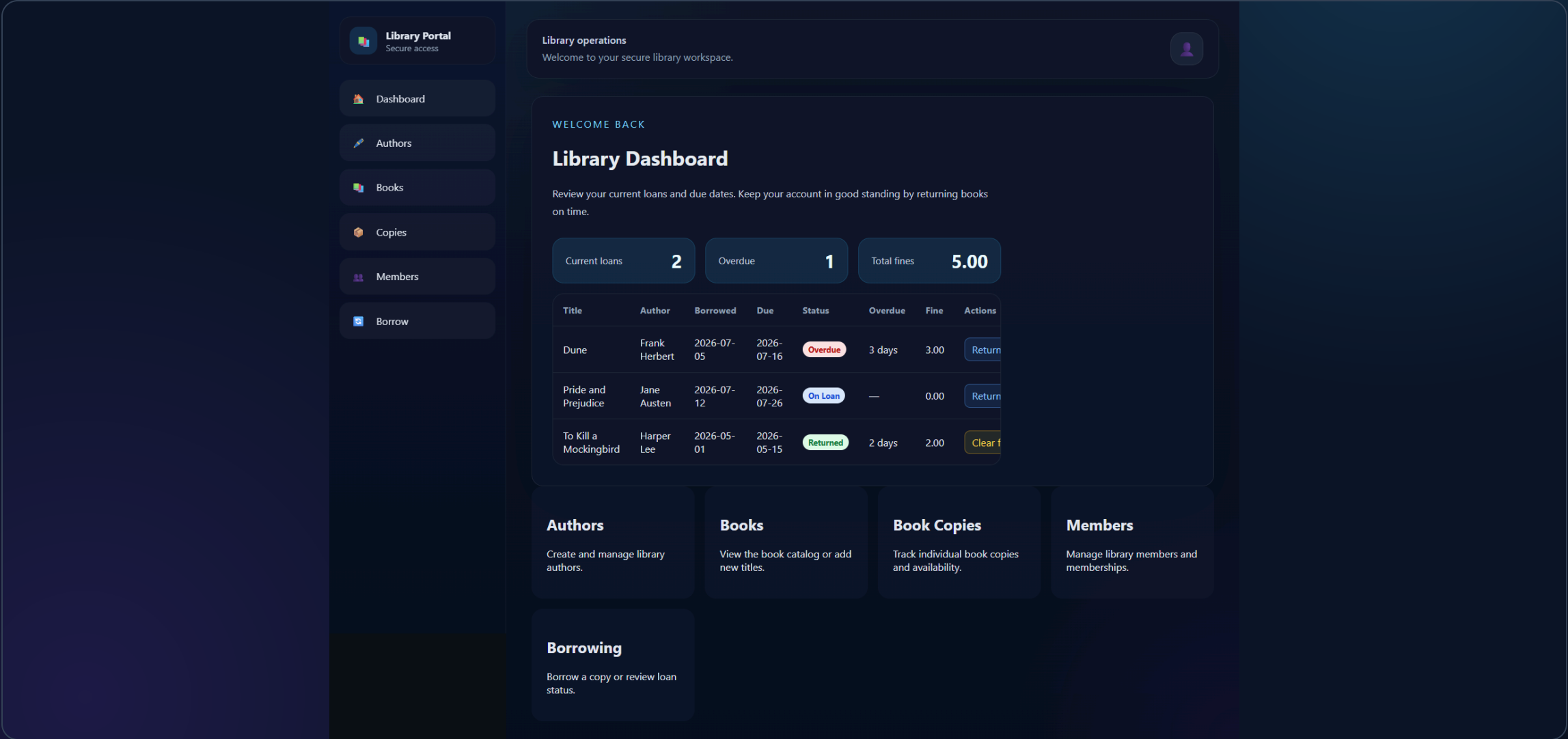
Capabilities follow the responsibility of each role

Capability	Student	Librarian	Administrator	Visitor
Browse books and availability	Yes	Yes	Yes	Yes
View personal dashboard	Yes	Yes	Yes	Authenticated view
Borrow / return own copies	Yes	Yes	Yes	No
Create authors and books	No	Yes	Yes	No
Register copies	No	Yes	Yes	No
Create/search members	No	Yes	Yes	No

Identity model

REST borrowing never trusts a client-provided member ID. It derives the member from the authenticated email and creates the linked member record on first borrow when required.

Personal dashboard: loans, due dates, returns & fines



Borrowing and returning update every downstream view



Borrow transaction

- Validates due date
- Selects first available physical copy
- Creates the borrow record
- Sets copy status to borrowed

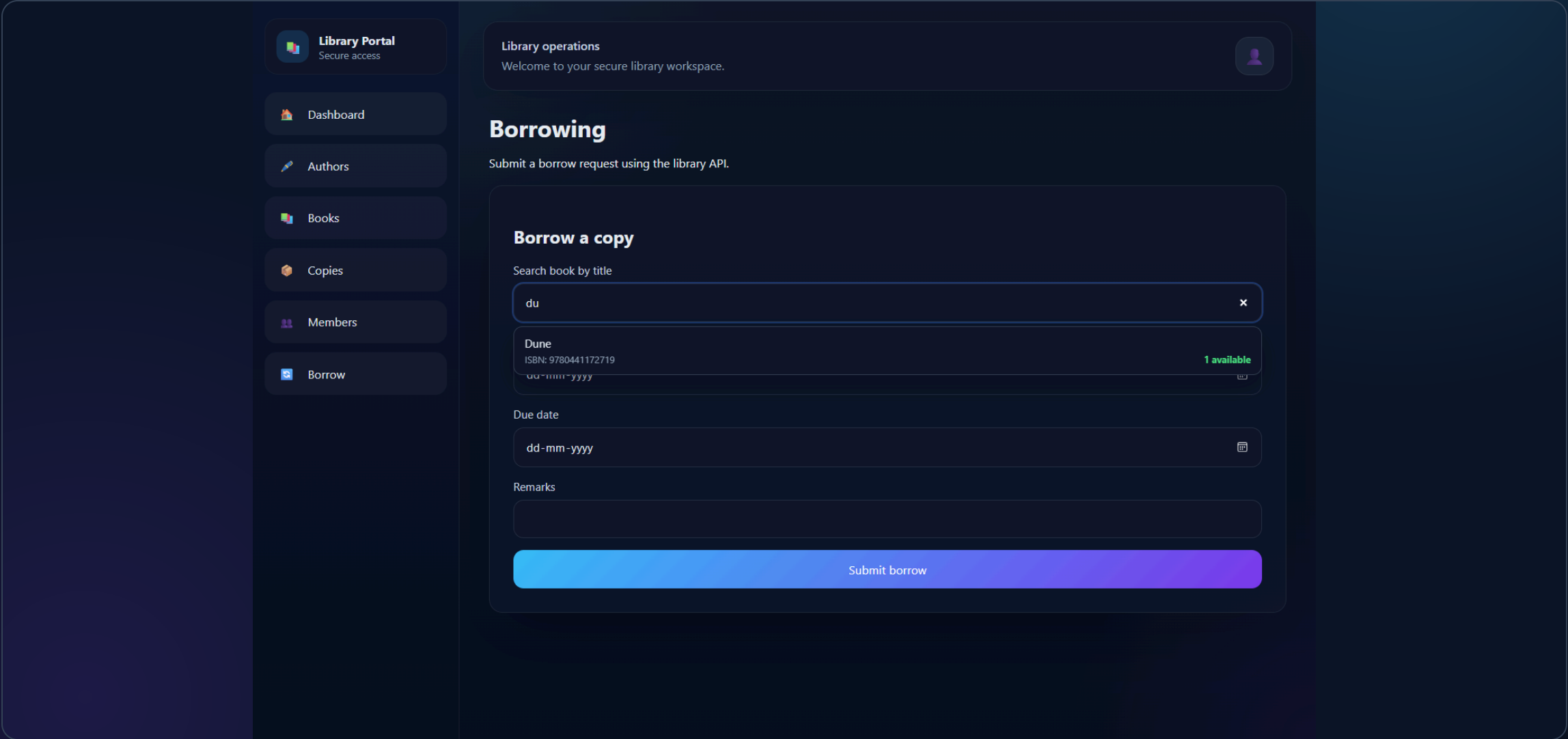
Return transaction

- Verifies record ownership
- Records today's return date
- Finalizes overdue fine
- Sets copy status to available

Fine rule

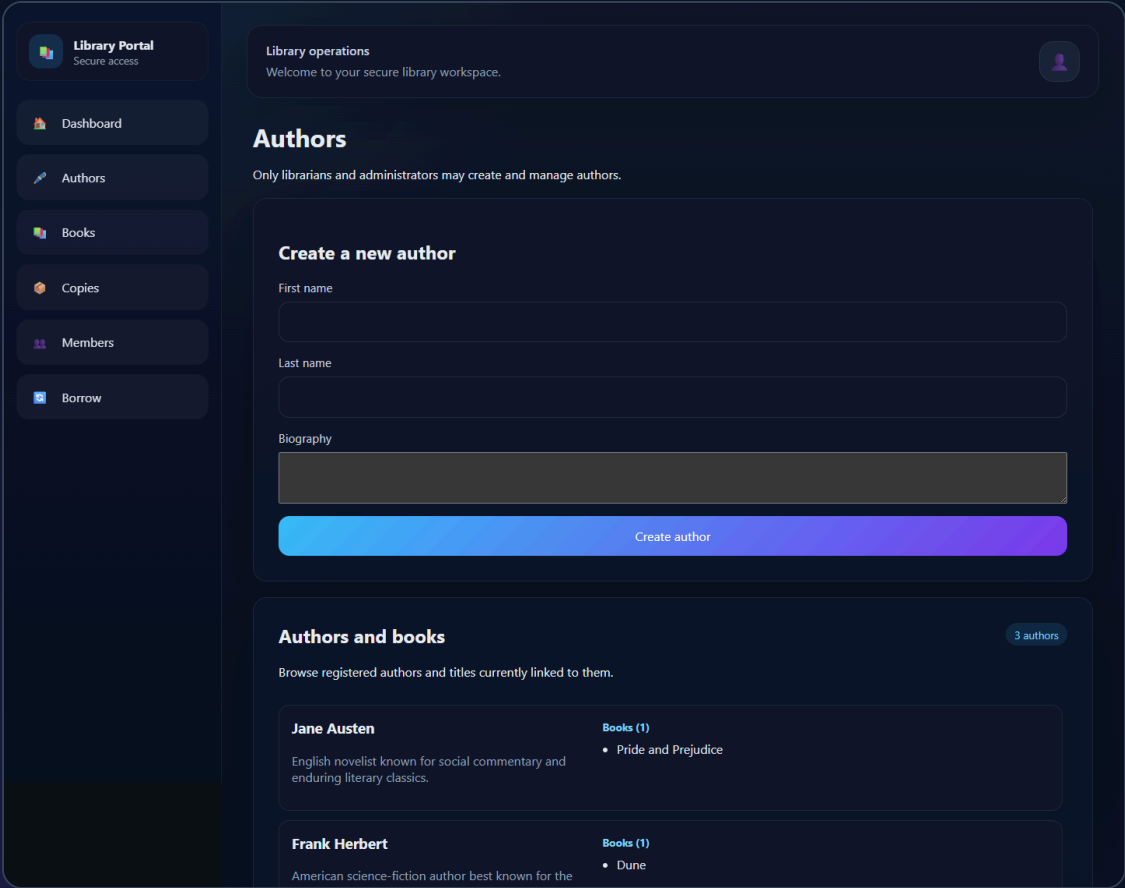
Outstanding fine = overdue days × 1.00. Active overdue fines are calculated dynamically; returned-loan fines stop accruing and can be cleared.

Borrow by title with inline live availability

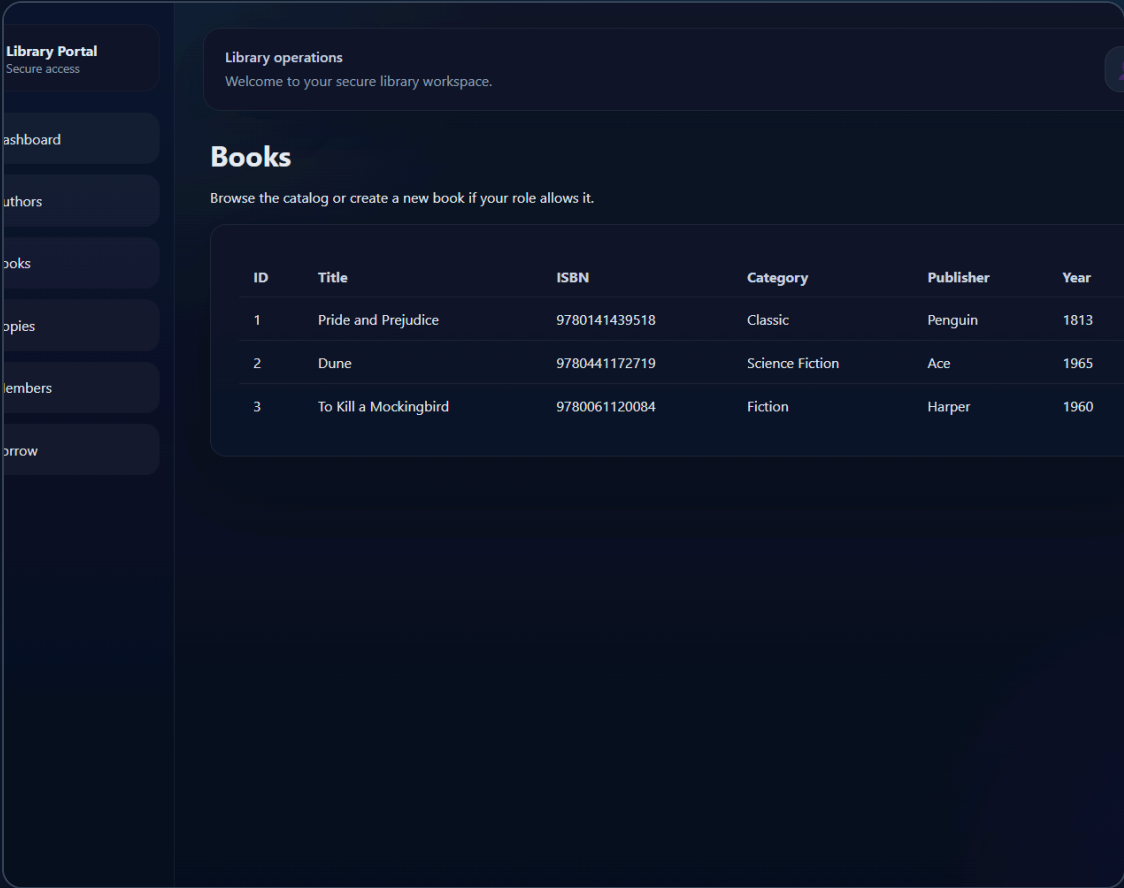


Catalog and author relationships

Authors with linked titles

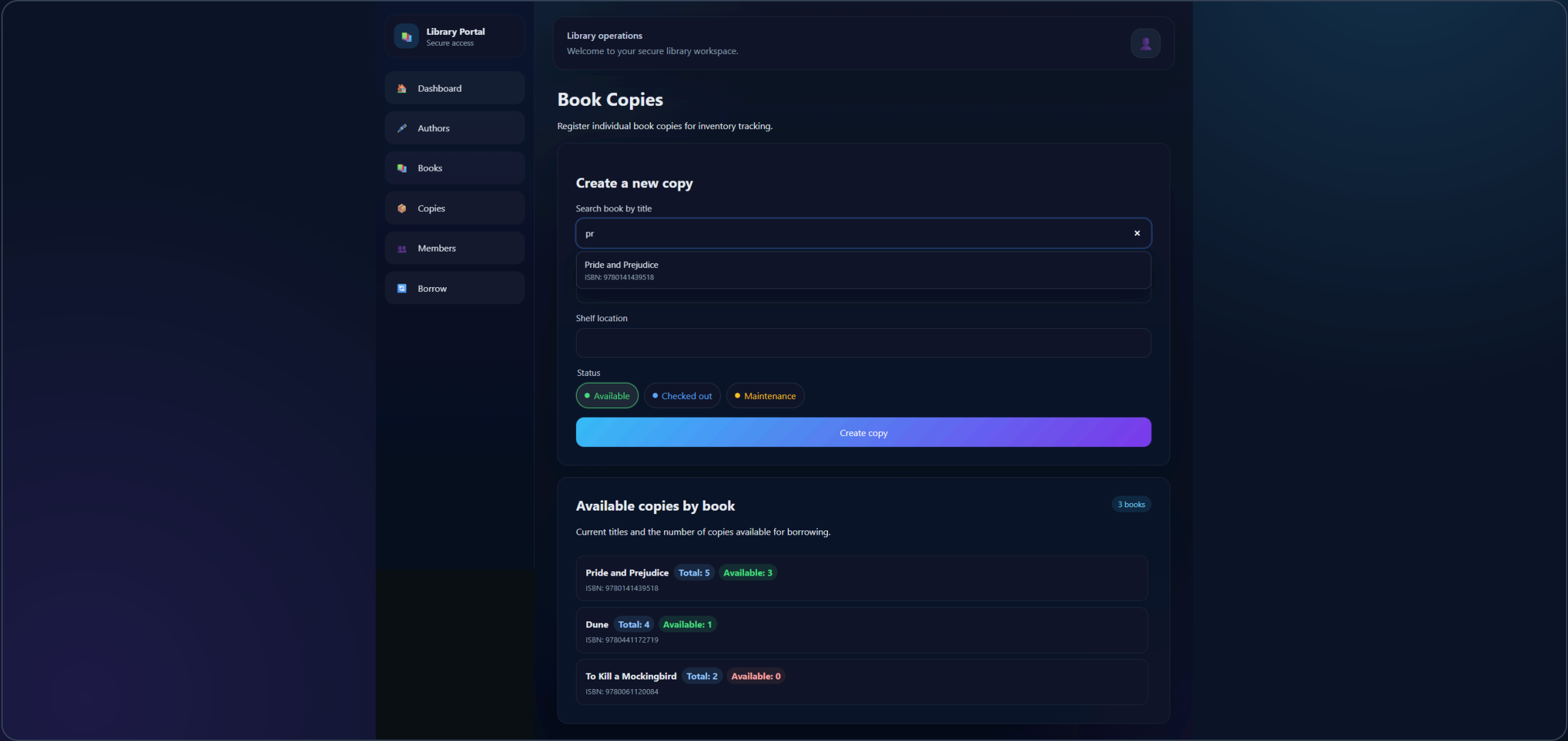


Book catalog



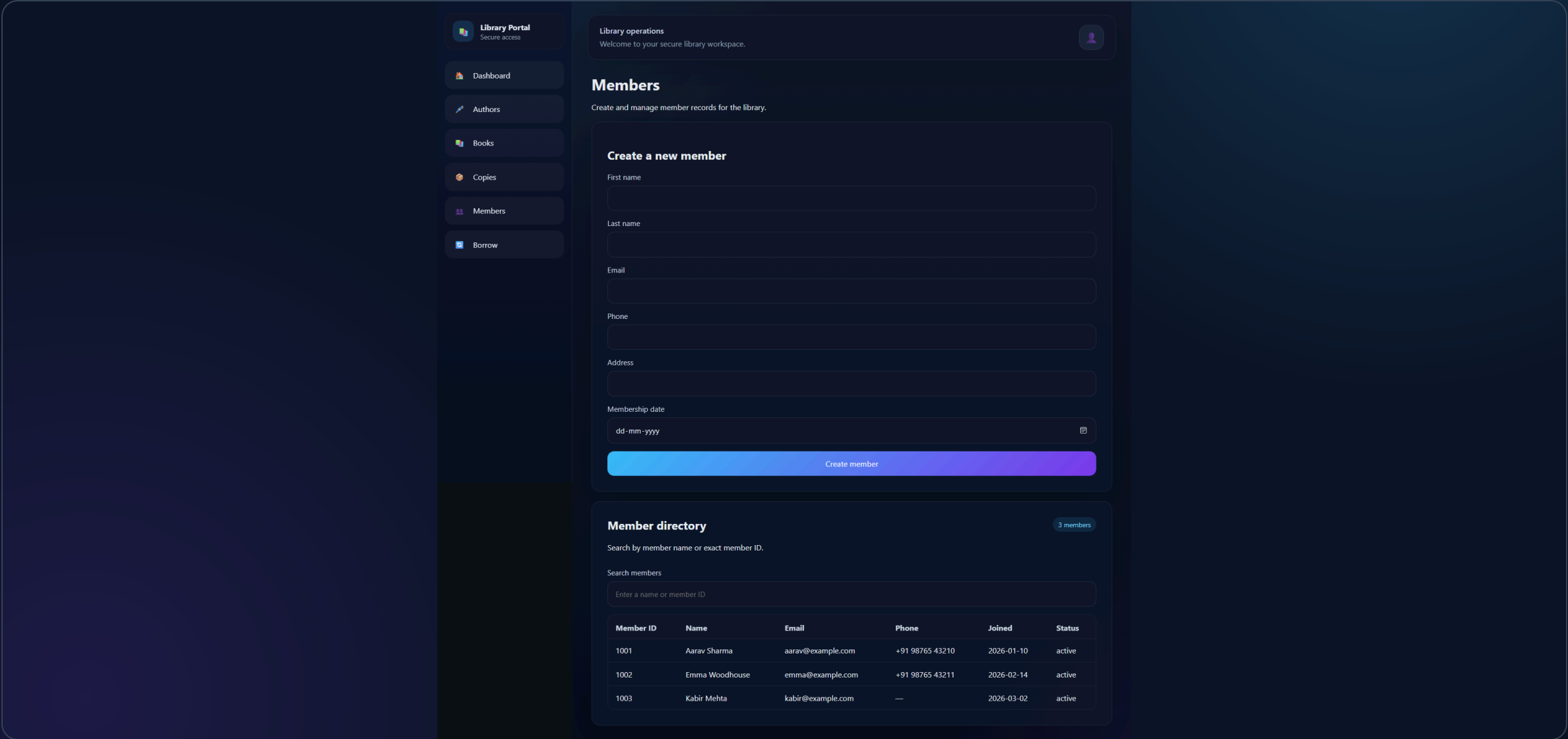
The clickable **Books (n)** relationship opens the catalog filtered by `author_id`.

Copy registration and availability intelligence



Title autocomplete · status picker · total and available counts

Searchable member operations



Interactive API documentation

Swagger UI

`http://localhost:8000/docs`

- Explore every operation and schema
- Use **Authorize** with a bearer JWT
- Submit requests and inspect live responses
- Review validation requirements

Additional contracts

`/redoc` provides reference-oriented documentation.

`/openapi.json` exposes the machine-readable OpenAPI document for code generation and integrations.

1. POST /token



2. Copy access_token



3. Swagger Authorize



4. Execute protected APIs

Authorization: Bearer <access_token>

Authentication, authors, books & inventory

Method	Path	Purpose	Access / parameters
GET	/health	Service health	Public
POST	/token	Authenticate and return JWT + roles	Public; email/password body
GET	/authors	Alphabetical author list	Authenticated
POST	/authors	Create author	Librarian/admin
GET	/books	List and search catalog	search, author_id
POST	/books	Create bibliographic record	Librarian/admin
GET	/books/{book_id}	Get one book	Authenticated
GET	/books/availability	Total and available copies per title	Optional search
POST	/copies	Create barcode-level copy	Librarian/admin

Members, borrowing, returns & fines

Method	Path	Purpose	Rules
GET	/members	List/search members	Librarian/admin; name or exact ID search
POST	/members	Create member	Librarian/admin; unique email
POST	/borrow	Borrow by book or copy	Member comes from authenticated email
GET	/me/borrows	Personal loan history	Includes status, overdue days, fine, title, author
POST	/me/borrows/{id}/return	Return owned loan	Finalizes fine and restores availability
POST	/me/borrows/{id}/clear-fine	Clear returned-loan fine	Loan must be returned and owned by caller

Ownership protection

The path ID alone is insufficient. The service filters by both `borrow_id` and the member resolved from the current token, preventing cross-member return or fine operations.

Borrow API in detail

Request

```
POST /borrow
Authorization: Bearer <token>
Content-Type: application/json

{
  "book_id": 2,
  "borrow_date": "2026-07-19",
  "due_date": "2026-08-02",
  "remarks": "Optional note"
}
```

Successful response

```
{
  "borrow_id": 41,
  "copy_id": 7,
  "member_id": 1001,
  "borrow_date": "2026-07-19",
  "due_date": "2026-08-02",
  "return_date": null,
  "fine_amount": 0,
  "remarks": "Optional note"
}
```

- Use either `book_id` or `copy_id`.
- `member_id` is deliberately excluded from the request.
- The service rejects a past due date relative to the borrow date and rejects unavailable inventory.

Predictable contracts for client applications

Borrow history enrichments

- status
- overdue_days
- fine_amount
- book_title
- book_author
- return_date

Canonical error

```
{
  "error": {
    "code": "invalid_data",
    "message": "Due date cannot be before the borrow date",
    "details": {
      "borrow_date": "2026-07-19",
      "due_date": "2026-07-18"
    }
  }
}
```

401 unauthenticated

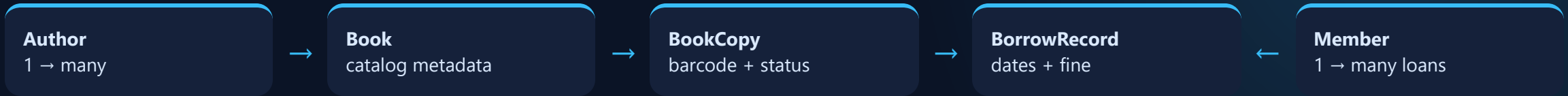
403 forbidden

404 not found

409 conflict

422 validation

Relational foundation



Authentication domain

User ↔ user_roles ↔ Role separates login authorization from library membership while linking them through email in REST self-service workflows.

Integrity rules

Unique user/member emails, ISBNs, API keys, role names, and copy barcodes prevent ambiguous identities and inventory records.

Verified feature coverage

12

BDD scenarios

66

Passing steps

0

Failures

Covered

Catalog search/filter, availability, member search, auto-linking, borrowing validation, overdue fines, returns, clearance, ownership, authentication, RBAC, HTTP flow, conflicts, and missing records.

Isolation

Behave sets TEST_DATABASE_URL to SQLite, recreates test tables, seeds test identities, and never clears the configured development PostgreSQL database.

```
cd backend
behave --no-capture
```

```
cd ../frontend
npm run build
```

SUMMARY

Ready for library operations

The platform connects discoverable catalog data, real copy availability, secure self-service circulation, and operational member tools through a documented and tested API.

Explore

Swagger: [/docs](#)

Integrate

OpenAPI: [/openapi.json](#)

Verify

`behave --no-capture`

LIBRARY